C++

(like your life depends on it)

Tomaz Canabrava / Tarcisio Fisher
(on behalf of `KDE` and `Codethink`)

Specially done for the Awesome Peeps at the
Univesity of Macedonia

What we are going to do here?

You already know how to Program.
We will not teach you that.

Cool, But then, What I'll learn?

How to not depend on an IDE
How to **not** create code from scratch

Starting from the Start

C++ Learning

- Multi-File C++ Projects
- External Libraries
- Creating Libraries (and sharing it)
- Finding Speed Problems
  - Optimizing It

But Why C++ and not $RAND_LANG

Because $RAND_LANG depends in C++. Seriously.

- C? Made with C++.
- C++? Made with C++.
- Rust? uses LLVM, Made with C++.
- Python? Made C, That uses C++.
- PHP? Made in C, That uses C++.
- Java? Made in C, C++.
- Swift? Made in C, C++.

C is made with C++?

Yes.

- GCC was ported to C++ in 2012
- LLVM is written in c++
- Clang (even for C!) is written in C++

So C++ is the backbone of everything

And no one will port a quadrillion of lines of code to another language just because.

Needed Software:

- CMake
- A C++ Compiler that can handle C++20
  - (GCC, Clang or Microsoft Visual Studio 2022)
- A Command Line Interface (No, I won't use an IDE)

if you have windows please use WSL

Let's start to Cry:

Hello World

Hello World?

- `mkdir` creates directories
- `touch` creates files

Create the follwing Folder Structure:

```
Project/
    src/
        CMakeLists.txt
        main.cpp
    CMakeLists.txt
    README.md
    LICENSE
```

Who are those files?

- `LICENSE` describes the license of your software
  - (A `lot` more about this later...)
- `CMakeLists.txt` describes what the compiler will do on this folder.
- `cpp` files are C++ sources.
- `README.md` will describe your project for humans

Project/CMakeLists.txt

- Toplevel CMakeLists.txt file
- Bundles the common logic of building the software
- Defines variables, and options, conditional compilation, etc.

```
cmake_minimum_required(VERSION 3.16 FATAL_ERROR)

project(CodeVis LANGUAGES CXX VERSION 0.1)

add_subdirectory(src)
```

Project/src/CMakeLists.txt

A CMakeLists.txt file inside of a folder means that it will build the inner files. In our case we have just a `main.cpp` file that will print `hello world`, so this file will be really small:

```
add_executable(hello_world main.cpp)
```

And Project/src/main.cpp

```cpp
#include <iostream>

int main() {
    std::cout << "Hello World";
    return 0;
}
```

Now, `what's wrong on this?`

(because, yes, there's something wrong.)

Let's compile and test:

```
➜  $ cmake -B build # Generates the build configuration
➜  $ cmake --build  build # Builds the system
➜  hello_world git:(master) ✗ ./build/src/hello_world
Hello World%
```

See the error?

And what `return 0` means?

C++ can use (some) C Headers

- let's import `<cstdlib>` to use `EXIT_SUCCESS`
- let's finish the `Hello World` cout with `\n`

```
#include <iostream>
#include <cstdlib>
int main() {
    std::cout << "Hello World\n";
    return EXIT_SUCCESS;
}
```

Why I didn't use std::endl? What's the difference with \n?

Compile and Test.

We should always use smaller entities

- Functions
- Structs
- Methods

Just like the good teacher said

So let's rewrite this to use a function `hello_world` that will be called on main

```cpp
#include <iostream>
#include <cstdlib>

// And this? Anything Wrong with it?
std::string hello_world() {
    return "hello world";
}

int main() {
    std::cout << hello_world() << "\n";
    return EXIT_SUCCESS;
}
```

an `std::string` is a value type that own's it's data.

That means that *each time* `hello_world()` is called, a `memory allocation` will happen.

Memory Allocations are expensive, and a `lot` of times you don't need it.

Let's look at this nice summary of the `assembly` of *just* the one liner that is `hello_world`

This is the generated machine code for

```
std::string hello_world() { return "hello world" }
```

```
hello_world[abi:cxx11]():
        push    rbp
        mov     rbp, rsp
        push    rbx
        sub     rsp, 40
        mov     QWORD PTR [rbp-40], rdi
        lea     rax, [rbp-25]
        mov     QWORD PTR [rbp-24], rax
        nop
        nop
        lea     rdx, [rbp-25]
        mov     rax, QWORD PTR [rbp-40]
        mov     esi, OFFSET FLAT:.LC0
        mov     rdi, rax
        call    std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> >::basic_string<std::allocator<char> >(char const*, std::allocator<char> const&)
        lea     rax, [rbp-25]
        mov     rdi, rax
        call    std::__new_allocator<char>::~__new_allocator() [base object destructor]
        nop
        jmp     .L10
        mov     rbx, rax
        lea     rax, [rbp-25]
        mov     rdi, rax
        call    std::__new_allocator<char>::~__new_allocator() [base object destructor]
        nop
        mov     rax, rbx
        mov     rdi, rax
        call    _Unwind_Resume
```

(you should see what python, java, php generates, hint... it's worse)

How to make it better?

Think about what you want.

Will your string be constant?
Will you need to `change` your string?
Do you know your value at `compile time`?
Do you must ensure `smallest` binary size?

lifetime-aware data?

If you are *looking* through a string, there's no need to copy.
The data inside of the `hello_world()` call is static, it will never change.
so `std::string_view` the hell out of it.

```cpp
#include <iostream>
#include <cstdlib>

std::string_view hello_world() {
    return "hello world";
}

int main() {
    std::cout << hello_world() << "\n";
    return EXIT_SUCCESS;
}
```

The assembly generated by

```
std::string_view hello_world() { return "hello world"; }
```

```
hello_world():
        push    rbp
        mov     rbp, rsp
        sub     rsp, 16
        lea     rax, [rbp-16]
        mov     esi, OFFSET FLAT:.LC0
        mov     rdi, rax
        call    std::basic_string_view<char, std::char_traits<char> >::basic_string_view(char const*) [complete object constructor]
        mov     rax, QWORD PTR [rbp-16]
        mov     rdx, QWORD PTR [rbp-8]
        leave
        ret
```

Way better

Can you do better?

Sure. but everything comes with a cost.

```
#include <iostream>
#include <cstdlib>

const char* hello_world() {
    return "hello world";
}

int main() {
    std::cout << hello_world() << "\n";
    return EXIT_SUCCESS;
}
```

This is the assembly generated with a `const char*`

```
hello_world():
        push    rbp
        mov     rbp, rsp
        mov     eax, OFFSET FLAT:.LC0
        pop     rbp
        ret
.LC1:
```

Pretty Neat right? But unsafe. Well, Kinda.

What to use, and when:

- Use data types that owns the data when you will modify the data
- Use `views` for data when you will just iterate and look to the data
  - std::span (for vectors, arrays, strings)
  - std::string_view (for strings, specialized.)
- Use `raw memory` or when the `API` you are targeting doesn't accept the others
- Don't use `raw memory` for speed, unless you measured it, the unsafety kinda kills it.

Keep it `std::string_view`

But... LIBRARIES!

It's bad to develop your code on `main.cpp`, Reusability is important.
Let's implement a library for our `std::string_view hello_world()`

- Should It be just an `include` call?
- Should it be an Static Library?
- Should it be a Dynamic library?

OH NOES! So many things to consider.

Libraries and Librarians

What makes a programming library, a library?

- Is it code-reusability?
- Is it a binary counterpart with structs, methods and functions?
- Is it a `link-time` component to access programable things?
- is it everything, and none, at the same time?

The correct answer is

`Everything and None at the same time`

Because it can be all of those things. There's no general rule.

Let's explore a bit how to create, and when to use.

Let's move the implementation of `hello world` to a library.

File `hello_world.h`

```
#pragma once

#include <string_view>

namespace HelloWorld {
    std::string_view hello() {
        return "hello world"
    }
}
```

And let's use the header like so:

File `main.cpp`

```cpp
#include <hello_world.h>
#include <iostream>

int main() {
    std::cout << HelloWorld::hello() << "\n";
}
```

So, is this a library?

Well, Yeah.

Do we need to do anything else?

Well, No.

So Easy!

Not really.

Drawbacks:

- Slower Compile Times
  - Every C++ source that uses this library will recompile the code for the library.
- Slower Link Times
  - Every compiled object file will need to strip the call, so that only one call remain.

So, is this library style used?

Hell yes.

Because for some things it makes sense.

Not for this.

Breaking Header and Impl

File `hello_world.h`

```
#pragma once

#include <string_view>

namespace HelloWorld {
    std::string_view hello();
}
```

Noted the difference?

File `hello_world.cpp`

```cpp
#include <hello_world.h>

namespace HelloWorld {
    std::string_view hello() {
        return "hello world";
    }
}
```

Now the implementation is on it's own cpp file,
meaning it will `not` recompile the same code all the time
`speeding up` compilations.

Try!

And error?

`couldn't find a definition of HelloWorld::hello` during link time.

OH NOES.

What happened?

compilling hello_world.cpp

file `src/CMakeLists.txt`

```
add_executable(hello_world main.cpp hello_world.cpp)
```

Now, everything works.

Is `hello_world.h` still a library?

No. It's Not.

A library should be able to be used from multiple binaries. Because we linked the `object file` generated by hello_world.cpp with the same `object_file` generated by main.cpp, we created a `single` binary, and we can't call `hello_world` from outside of it making it `not` a library.

Static Libraries!

file `src/CMakeLists.txt`

```
add_library(hello_lib STATIC hello_world.cpp)

# Quite important. Always Aliases libraries.
add_library(MyLibraries::hello ALIAS hello_lib)

add_executable(hello_world main.cpp)

target_link_libraries(hello_world MyLibraries::hello)
```

It Compiles!

And the binaries it generates?

`hello_lib.a` (or `.lib` on windows)
`hello_world` executable.

And now you can share the `hello_lib.a` together with the `header` so more people can use your library.

Ok, Let's organize this library Better

Things to think about:

- Who will use this library?
- What is the public facing API?
- What's the private API? How do I hide it?
- Should I be concerned with API stability?
- What about ABI stability?

Jokes aside it's a library to write Hello World.
Let's prepare the CMake for a `real world` usage.

```
hello_lib\
    src\
      CMakeLists.txt
      hello_world.h
      hello_world.cpp
      hello_world.t.cpp
UniMacedoniaConfig.cmake.in
CMakeLists.txt
```

Questions:

- Why no more `main.cpp`?
- What's the `.t.cpp` ?
- What will change in the main `CMakeLists.txt` file?

CMake for the Library

I Want to:
- Use the library in my own tree
- Use the library somewhere else
- Allow to install and distribute the library

This will be complex, and broken in multiple slides.

`root/CMakeLists.txt` - First Slide, The easy one.

```
cmake_minimum_required(VERSION 3.16 FATAL_ERROR)

project(CodeVis LANGUAGES CXX VERSION 1.0)

# For instalation
include(GNUInstallDirs)

# To create the PackageConfig procedure
include(CMakePackageConfigHelpers)

# To be able to hide private information on the library.
include(GenerateExportHeader)

add_subdirectory(src)
```

`second slide` the start of the complexity.

```
write_basic_package_version_file(
  ${CMAKE_CURRENT_BINARY_DIR}/UniMacedoniaConfigVersion.cmake
  VERSION 1.0
  COMPATIBILITY SameMajorVersion)

configure_package_config_file(
    UniMacedoniaConfig.cmake.in
    ${CMAKE_CURRENT_BINARY_DIR}/UniMacedoniaConfig.cmake
    INSTALL_DESTINATION lib/UniMacedonia/cmake
)
```

What's a `configure_file` and it's specializations?

A `configure_file` is a cmake function that will act as a template engine, working through an `input file` and creating `another file` as output, on the `build` folder. We can use that to generate `sources` that will be consumed in build time, but we can also use that for anything else.

configure_package_config_file?

The `configure_package_config_file` function removes the complexity of creating the Config files by hand, with minimal (but annoying) intervention.

We don't know yet what we will have as a library, but we already know it will consume a `template` file, and generate a `UniMacedoniaConfig.cmake` that can be used later for other projects.

And our Template File?

`UniMacedoniaConfig.cmake.in`

```
@PACKAGE_INIT@

include("${CMAKE_CURRENT_LIST_DIR}/UniMacedoniaTargets.cmake")

check_required_components(UniMacedonia)
```

Evertything between the `@`'s will be replaced by CMake. This is a `base` version and some complex libraries will add more things to it. But it's ok for a medium-sized library.

Back to our main CMakeLists.txt

Now, we need to act on the `install` command, to move those files to their correct place.

```
install(FILES
    ${CMAKE_CURRENT_BINARY_DIR}/UniMacedoniaConfig.cmake
    ${CMAKE_CURRENT_BINARY_DIR}/UniMacedoniaConfigVersion.cmake
DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/UniMacedonia )

install(EXPORT UniMacedoniaTargets
        FILE UniMacedoniaTargets.cmake
        NAMESPACE UniMacedonia::
        DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/UniMacedonia
)
```

UniMacedoniaTargets.cmake?

Yes. CMake can export all the targets you used on a project automatically to a file that will be consumed by the Config file later.

And the Library CMake?

`src/hello_lib/CMakeLists.txt` (Part 1)

```
add_library(hello_lib SHARED hello_world.cpp)
add_library(UniMacedonia::hello_lib ALIAS hello_lib)

generate_export_header(hello_lib)

target_include_directories(hello_lib PUBLIC
  $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}>
  $<INSTALL_INTERFACE:include/UniMacedonia>
)
```

`src/hello_lib/CMakeLists.txt` (Part 2)

```
SET(HELLO_LIB_PUB_HEADERS
    hello_world.h
    ${CMAKE_CURRENT_BINARY_DIR}/hello_lib_export.h
)

set_target_properties(hello_lib PROPERTIES
  PUBLIC_HEADER "${HELLO_LIB_PUB_HEADERS}"
)

install(TARGETS hello_lib
  EXPORT UniMacedoniaTargets
  LIBRARY DESTINATION ${CMAKE_INSTALL_LIBDIR}
  ARCHIVE DESTINATION ${CMAKE_INSTALL_LIBDIR}
  RUNTIME DESTINATION ${CMAKE_INSTALL_BINDIR}
  PUBLIC_HEADER DESTINATION "${CMAKE_INSTALL_INCLUDEDIR}/UniMacedonia")
```

And that's it.

With this, and a `make && make install`, your library is ready to be consumed by other people, and other projects that you do.

The Header and C++ files will continue the same

Let's Consume it, from a new `project`.

Just a really small example this time, to consume the library we created and installed in the system. no fancy folder structure.

```
➜  hello_world_using_lib git:(master) tree
.
├── CMakeLists.txt
└── main.cpp
```

Our CMakeFile now will look for the library we just created, and link with the Binary.

```
cmake_minimum_required(VERSION 3.16 FATAL_ERROR)

project(HelloWorld LANGUAGES CXX VERSION 0.1)

find_package(UniMacedonia REQUIRED)

add_executable(hello_world_with_lib main.cpp)
target_link_libraries(hello_world_with_lib UniMacedonia::hello_lib)
```

And our main.cpp will just use as usual:

```cpp
#include <hello_world.h>

#include <iostream>

int main() {
    std::cout << UniMacedonia::hello_world() << "\n";
}
```

Possible Errors:

- CMake can't find the library
  - export UniMacedonia_DIR with the dir of the CMakeFiles
  - export UniMacedonia_DIR=/Data/lib/cmake/UniMacedonia
- The library is found but linking fails
  - C++ lacks `defaults`.
    - We need to make our own defaults, using `visibility rules`

Visibility errors looks like this:

```
[ 50%] Linking CXX executable hello_world_with_lib
/usr/bin/ld: CMakeFiles/hello_world_with_lib.dir/main.cpp.o: in function `main':
main.cpp:(.text+0xa): undefined reference to `UniMacedonia::hello_world()'
collect2: error: ld returned 1 exit status
make[2]: *** [CMakeFiles/hello_world_with_lib.dir/build.make:98: hello_world_with_lib] Error 1
make[1]: *** [CMakeFiles/Makefile2:83: CMakeFiles/hello_world_with_lib.dir/all] Error 2
make: *** [Makefile:91: all] Error 2
```

Visibility Rules

- What you want to allow the user to see you export.
- what you don't, you do nothing.
- Let's force CMake to hide everything
- Let's allow only what we want

So, Let's default everything to hidden, and export only what we want.

file `src/hello_lib/CMakeLists.txt`

```
set_target_properties(hello_lib PROPERTIES CXX_VISIBILITY_PRESET hidden)
```

file `src/hello_lib/hello_world.h`

```
#pragma once

#include <string_view>
#include <hello_lib_export.h>

namespace UniMacedonia {
    HELLO_LIB_EXPORT std::string_view hello_world();
}
```

Uffs, 60 slides.

We Learned:

- How to Create Programs
- How to Create Libraries
  - How To Export Libraries
  - How to Consume Libraries
  - How to Hide Internal API
- We cried too.

Thanks for testing the most difficult Hello World Ever.

This is not C++

- Sorry. It is.

You need to know how to distribute, and build your projects, binaries, libraries, so you can use multiple pre-written libraries, and also create your own.

Recap:

- use CMake, or at least something that generates CMake find packages for your libraries.
- Prepare your code as a library from the start, an application should be small and contained into a single .m.cpp file
- use the CMake call `generate_export_header` to generate an export header for your library
- Don't reinvent the wheel, prepare code for reusability.

Questions?

- Will the next lesson have *more* actual C++?
  - Now that we know how to create the base project, yes.
- Don't you have any shame?
  - No.

Contact

Tomaz Canabrava

- tcanabrava@kde.org
- tcanabrava@archlinux.org